

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	M.S.Dinesh	Reg. No.:	192424016
Section / Batch:	I	Date:	22.08.206

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

Code of Conduct

I (M.S.Dinesh / 192424016) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: M.S.Dinesh

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
----------	-----------	---------------------	----------------------	----------------------	---------------------

Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Q1 E-Commerce Recommendation Optimization (Graph-Based Ranking)

An online platform must recommend products to users based on browsing history and similarity patterns. Analyze how graph-based algorithms can be used to model user-product relationships. Identify constraints such as large-scale data and dynamic updates. Design a solution using ranking algorithms and similarity measures. Discuss implementation using scalable tools. Evaluate algorithm performance and efficiency. Interpret the impact on user engagement and business growth.

Solution

Introduction & Graph Modeling

- **Bipartite Structure:** Models interaction data using a bipartite graph containing two distinct node sets: Users (U) and Products (P).
- **Heterogeneous Edges:** Captures multi-modal interactions (clicks, add-to-cart, purchases) as directed, weighted edges reflecting user intent strength.
- **Implicit Connections:** Derives user-user and product-product similarity edges through projecting the bipartite graph into unipartite sub-graphs.
- **Cold-Start Handling:** Links content metadata attributes (categories, brands) as additional nodes to bridge disconnected entities.
- **Structural Context:** Captures high-order neighborhood relationships that traditional collaborative filtering matrices completely miss.

Solution Implementation (Python & NetworkX)

This scalable prototype uses **Personalized PageRank (Random Walk with Restart)** to score and rank product recommendations for a specific target user.

python

```
import networkx as nx
```

```

def generate_graph_recommendations(user_id, interaction_data, top_k=5):

    # 1. Initialize bipartite graph

    G = nx.Graph()

    # 2. Populate nodes and weighted edges

    for user, item, weight in interaction_data:

        G.add_edge(user, item, weight=weight)

    # 3. Handle edge-case: user not found

    if user_id not in G:

        return []

    # 4. Personalization vector: set target user anchor to 1.0

    personalization = {node: 0.0 for node in G.nodes()}

    personalization[user_id] = 1.0

    # 5. Execute Personalized PageRank (Alpha = damping factor)

    ppr_scores = nx.pagerank(G, alpha=0.85, personalization=personalization,
weight='weight')

    # Filter out non-product nodes and products the user already interacted with

    interacted_items = set(G.neighbors(user_id))

    recommendations = [

        (node, score) for node, score in ppr_scores.items()

        if str(node).startswith('p_') and node not in interacted_items

```

```
]
```

```
# Sort by score descending

recommendations.sort(key=lambda x: x[1], reverse=True)

return recommendations[:top_k]
```

```
# Mock Input: (User, Product, Weight)

data = [('u_1', 'p_A', 1), ('u_1', 'p_B', 3), ('u_2', 'p_B', 2), ('u_2', 'p_C', 5)]

print(generate_graph_recommendations('u_1', data))
```

Use code with caution.

Algorithm Steps

- **Graph Construction:** Ingests raw behavior streaming logs to construct or update nodes and topological edge links.
- **Personalization Setup:** Sets up a random walk probability vector anchored heavily on the active user's historical nodes.
- **Random Walk Iteration:** Simulates structural path traverses across edges, utilizing weights to bias paths toward higher-value actions like purchases.
- **Convergence Evaluation:** Runs power-iteration steps continuously until node probability vectors stabilize below a strict convergence delta.
- **Candidate Ranking:** Filters out visited items, isolates product nodes, and ranks remaining nodes by steady-state probability scores.

Mathematical Formulas

- **Personalized PageRank (PPR):**
$$r = (1 - \alpha)s + \alpha W r$$

(Where r is ranking vector, α is damping factor, s is personalization vector, and W is transition matrix)
- **Cosine Node Similarity:**
$$\text{Sim}(u_i, u_j) = \frac{\mathbf{N}(u_i) \cdot \mathbf{N}(u_j)}{\|\mathbf{N}(u_i)\| \|\mathbf{N}(u_j)\|}$$
- **Transition Probability:**

$$P(v_j | v_i) = \frac{\text{weight}(v_i, v_j)}{\sum_{k \in N(v_i)} \text{weight}(v_i, v_k)}$$

- **Time Complexity (Inference):**

$$O(k \cdot |E|)$$

(Where k is power-iterations and $|E|$ is active localized edge density)

- **Space Complexity (Graph Storage):**

$$O(|V| + |E|)$$

(Linear memory mapping proportional to total nodes and active connections)

Constraints & Scalable Tools

- **Data Volume Bottleneck:** Distributed Graph Neural Networks (GNNs) leverage cluster memories across multiple parallel processing server nodes.
- **Real-time Latency Limits:** Graph databases like **Neo4j** or **Amazon Neptune** maintain sub-millisecond multi-hop neighborhood traversals.
- **Dynamic Interaction Drift:** Stream processing via **Apache Spark GraphX** or **Flink** updates edge weights on the fly.
- **High-Dimensional Embeddings:** Frameworks like **PyTorch Geometric (PyG)** scale graph node vector representations to billions of parameters.
- **Memory Swapping Costs:** Negative-sampling pipelines optimize loss calculation loops, discarding irrelevant node relationships to save RAM.

Performance & Efficiency Advantages

- **High Serendipity:** uncovers non-obvious item connections across deep multi-hop links that normal search index tools miss.
- **Spam Resilience:** Biasing walks via localized personalization vectors nullifies broad system-wide gaming or artificial review pumping.
- **Sparse Matrix Recovery:** Performs accurately even when users have minimal click history by exploiting shared neighborhood connectivity structures.
- **Flexible Multi-Modal Integration:** Merges text embeddings, image tokens, and categorical metadata directly into standard graph topology mappings.
- **Explainable Paths:** Provides trace routes explaining *why* an item was chosen (e.g., "User A bought Item X → Item Y").

Engagement & Business Impact

- **Click-Through Rate (CTR):** Boosts user exploration metrics by surfacing contextually hyper-relevant items on home feeds.
- **Conversion Rate Lift:** Directs active shoppers straight to target items, cutting browsing

friction and cart abandonment rates.

- **Average Order Value (AOV):** Drives cross-selling initiatives by recommending companion products using structural unipartite projection rules.
- **Long-Tail Discovery:** Exposes less-popular inventory items to niche audiences, reducing clearance storage costs significantly.
- **Customer Retention (LTV):** Increases habituation metrics by generating refreshing, dynamic content recommendations upon every session refresh.

Output

**SIMATS**
ENGINEERING

SIMATS Online Programming Lab
DAA PROGRAMMING

DINESH M S
192424016RC

Language: Python C C++ Java

QUESTIONS

Q1
E-Commerce Recommendation Optimization (Graph-Based Ranking)
100 marks

1/1 answered

Q1 E-Commerce Recommendation Optimization (Graph-Based Ranking) 100 marks

An online platform must recommend products to users based on browsing history and similarity patterns. Analyze how graph-based algorithms can be used to model user-product relationships. Identify constraints such as large-scale data and dynamic updates. Design a solution using ranking algorithms and similarity measures. Discuss implementation using scalable tools. Evaluate algorithm performance and efficiency. Interpret the impact on user engagement and business growth.

Your Code

```
1 import math
2 from collections import defaultdict
3
4 user_products = {
5     "U1": {"P1", "P2", "P3"},
6     "U2": {"P2", "P4"},
7     "U3": {"P1", "P4", "P5"},
8 }
9 def jaccard(a, b):
10     inter = len(a & b)
11     union = len(a | b)
12     return inter / union if union else 0.0
13
14 product_users = defaultdict(set)
15 for u, ps in user_products.items():
16     for p in ps:
17         product_users[p].add(u)
18
19 def recommend(user, top_k=3):
20     interacted = user_products.get(user, set())
21     candidates = set()
22     for p in interacted:
23         candidates |= set(product_users[p].keys())
```

Input
stdin input

Output
Recommendations for U1: ['P4', 'P5']